

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (**BL 5**)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I X. Joshua – (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

1. K-Nearest Neighbour (KNN)

```
import math

X = [(1, 1), (2, 2), (3, 3), (6, 6), (7, 7), (8, 8)]
y = ['A', 'A', 'A', 'B', 'B', 'B']
test = (5, 5)

k = int(input("Enter K: "))

dist = []

for i in range(len(X)):
    d = math.sqrt((X[i][0]-test[0])**2 + (X[i][1]-test[1])**2)
    dist.append((d, y[i]))

dist.sort()

neighbors = dist[:k]

classes = [c for d, c in neighbors]

prediction = max(set(classes), key=classes.count)

print("Predicted Class:", prediction)

correct = sum(1 for i in range(len(X))
    if max(set([c for d, c in sorted([
        (math.sqrt((X[j][0]-X[i][0])**2 +
            (X[j][1]-X[i][1])**2), y[j])
        for j in range(len(X)) if j != i
    ][:k])), key=[c for d, c in sorted([
        (math.sqrt((X[j][0]-X[i][0])**2 +
            (X[j][1]-X[i][1])**2), y[j])
        for j in range(len(X)) if j != i
    ][:k]).count) == y[i])

print("Accuracy:", correct / len(X) * 100, "%")
```

Key idea: KNN calculates Euclidean distance and assigns the class occurring most often among the K nearest points.

2. Locally Weighted Regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 8, 10])

query = 3.5
tau = 1.0

# Calculate weights
weights = np.exp(-(X - query) ** 2) / (2 * tau ** 2)

# Weighted prediction
X1 = np.vstack([np.ones(len(X)), X]).T
theta = np.linalg.inv(X1.T @ np.diag(weights) @ X1) @ \
    (X1.T @ np.diag(weights) @ y)

prediction_lwr = theta[0] + theta[1] * query

# Standard Linear Regression
model = LinearRegression()
model.fit(X.reshape(-1, 1), y)
prediction_lr = model.predict([[query]])[0]

print("LWR Prediction:", prediction_lwr)
print("Linear Regression:", prediction_lr)
```

Key idea: Nearby data points receive larger weights.

3. Radial Basis Function (RBF) Network

```
import numpy as np
```

```
X = np.array([1, 2, 3, 4, 5])
```

```
y = np.array([2, 4, 6, 8, 10])
```

```
centers = np.array([1, 3, 5])
```

```
sigma = 1.0
```

```
def gaussian(x, c):
```

```
    return np.exp(-((x-c)**2) / (2*sigma**2))
```

```
# RBF activation matrix
```

```
Phi = np.array([[gaussian(x, c) for c in centers] for x in X])
```

```
# Train output weights
```

```
weights = np.linalg.pinv(Phi) @ y
```

```
# Prediction
```

```
test = 3.5
```

```
phi_test = np.array([gaussian(test, c) for c in centers])
```

```
prediction = phi_test @ weights
```

```
print("Predicted Output:", prediction)
```

```
print("Centers:", centers)
```

```
print("Width (sigma):", sigma)
```

Effect of parameters:

- Smaller sigma → narrow RBF and local learning.
 - Larger sigma → wider RBF and smoother predictions.
 - Good centers improve accuracy.
-

4. Case-Based Learning

```
import math
```

```
cases = [  
    ([1, 2], "Low"),  
    ([2, 3], "Low"),  
    ([7, 8], "High"),  
    ([8, 9], "High")  
]
```

```
query = [6, 7]
```

```
best_distance = float('inf')
```

```
best_solution = None
```

```
for features, solution in cases:
```

```
    d = math.sqrt(sum((a-b)**2 for a, b in zip(features, query)))
```

```
    print(features, "Distance:", d)
```

```
    if d < best_distance:
```

```
        best_distance = d
```

```
        best_solution = solution
```

```
print("Most Similar Case Solution:", best_solution)
```

```
print("Similarity Distance:", best_distance)
```

Similarity calculation:

Euclidean distance is used:

Smaller distance means greater similarity.

5. KNN vs RBF

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.kernel_ridge import KernelRidge
```

```
X = np.array([[1], [2], [3], [7], [8], [9]])
y = np.array([0, 0, 0, 1, 1, 1])
```

```
test = np.array([[5]])
```

```
# KNN
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)
knn_pred = knn.predict(test)
```

```
# RBF
```

```
rbf = KernelRidge(kernel='rbf', gamma=0.5)
rbf.fit(X, y)
rbf_pred = rbf.predict(test)
```

```
print("KNN Prediction:", knn_pred[0])
```

```
print("RBF Prediction:", rbf_pred[0])
```

Comparison:

KNN	RBF
Uses nearest points	Uses Gaussian basis functions
Simple	More computationally involved
Prediction based on neighbors	Prediction based on learned weights
Sensitive to K	Sensitive to centers and width

6. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

```
X = [[1, 1], [2, 1], [3, 2], [7, 7], [8, 8], [9, 8]]
y = ['A', 'A', 'A', 'B', 'B', 'B']
```

```
model = DecisionTreeClassifier(criterion='gini', random_state=0)
model.fit(X, y)
```

```
test = [[5, 5]]
print("Predicted Class:", model.predict(test)[0])
print("Accuracy:", model.score(X, y) * 100, "%")
```

```
plot_tree(model, feature_names=['Feature1', 'Feature2'],
          class_names=['A', 'B'], filled=True)
plt.show()
```

Key idea: The tree selects features using **Gini Index** or **Information Gain**. A good feature gives better separation between classes.

7. Naïve Bayes with Laplace Smoothing

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
X = [[1, 20], [2, 25], [3, 30], [7, 70], [8, 75], [9, 80]]
y = ['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass']
```

```
model = GaussianNB()
model.fit(X, y)
```

```
test = [[6, 60]]
```

```
prediction = model.predict(test)
```

```
print("Predicted Class:", prediction[0])
```

```
print("Accuracy:", accuracy_score(y, model.predict(X)) * 100, "%")
```

Naïve Bayes uses:

Laplace smoothing prevents zero probability for unseen feature values.

8. Logistic Regression Using Gradient Descent

```
import numpy as np
```

```
X = np.array([1, 2, 3, 4, 5])
```

```
y = np.array([0, 0, 0, 1, 1])
```

```
w = 0.0
```

```
b = 0.0
```

```
lr = 0.1
```

```
def sigmoid(z):
```

```
    return 1 / (1 + np.exp(-z))
```

```
for epoch in range(1000):
```

```
    z = w * X + b
```

```
    p = sigmoid(z)
```

```
    dw = np.mean((p-y) * X)
```

```
    db = np.mean(p-y)
```

```
    w -= lr * dw
```

```
    b -= lr * db
```



```
probability = sigmoid(w * 4 + b)
prediction = 1 if probability >= 0.5 else 0
```

```
print("Sigmoid Output:", probability)
print("Predicted Class:", prediction)
```

```
pred = (sigmoid(w*X+b) >= 0.5).astype(int)
accuracy = np.mean(pred == y) * 100
print("Accuracy:", accuracy, "%")
```

Learning rate:

- Too small → slow convergence.
 - Too large → may overshoot the minimum.
 - Suitable learning rate → faster and stable convergence.
-

9. K-Means Clustering

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
X = np.array([
    [1, 1], [2, 2], [3, 3],
    [8, 8], [9, 9], [10, 10]
])
```

```
k = 2
```

```
model = KMeans(n_clusters=k, random_state=0, n_init=10)
labels = model.fit_predict(X)
```

```
print("Cluster Labels:", labels)
```

```

print("Centroids:")
print(model.cluster_centers_)

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(model.cluster_centers_[0],
            model.cluster_centers_[1],
            marker='X')
plt.show()

```

Steps:

1. Initialize centroids.
2. Assign points to nearest centroid.
3. Calculate new centroids.
4. Repeat until centroids stop changing.

10. Genetic Algorithm

```

import random

def fitness(x):
    return x * x

population = [random.randint(0, 31) for _ in range(10)]

for generation in range(20):

    population.sort(key=fitness, reverse=True)

    parents = population[:4]

    children = []
    while len(children) < 6:

```

```

p1 = random.choice(parents)
p2 = random.choice(parents)

child = (p1 + p2) // 2

# Mutation
if random.random() < 0.1:
    child += random.randint(-2, 2)
    child = max(0, min(31, child))
    children.append(child)
population = parents + children
print("Generation:", generation + 1,
      "Best:", max(population, key=fitness),
      "Fitness:", max(map(fitness, population)))
best = max(population, key=fitness)
print("Best Solution:", best)
print("Best Fitness:", fitness(best))

```

GA operations:

- Selection
- Crossover
- Mutation
- Fitness evaluation

Higher mutation may increase diversity but can slow convergence.

11. Support Vector Machine (SVM)

```

import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
X = np.array([
    [1, 1], [2, 2], [3, 3],

```

```

    [7, 7], [8, 8], [9, 9]
])

y = [0, 0, 0, 1, 1, 1]

model = SVC(kernel='linear')
model.fit(X, y)

test = [[5, 5]]
prediction = model.predict(test)

print("Predicted Class:", prediction[0])
print("Accuracy:", accuracy_score(y, model.predict(X)) * 100, "%")
print("Support Vectors:")
print(model.support_vectors_)

```

Key idea: SVM finds the separating hyperplane with the **maximum margin**.

Support vectors are the points closest to the decision boundary and strongly influence the classifier.

12. Principal Component Analysis (PCA)

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

```

```

X = np.array([
    [2, 4, 6],
    [3, 6, 9],
    [4, 8, 12],
    [8, 16, 24],
    [9, 18, 27],

```

```

    [10, 20, 30]
])

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Original Shape:", X.shape)
print("Reduced Shape:", X_pca.shape)
print("Variance Retained:", pca.explained_variance_ratio_)
print("Total Variance:", sum(pca.explained_variance_ratio_))

plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

```

Key idea: PCA reduces dimensions while retaining maximum variance.

13. Hierarchical Clustering

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

X = np.array([
    [1, 1], [2, 2], [3, 3],
    [8, 8], [9, 9], [10, 10]
])

for method in ['single', 'complete', 'average']:

```

```
Z = linkage(X, method=method)
```

```
plt.figure()
```

```
dendrogram(Z)
```

```
plt.title(method + " Linkage")
```

```
plt.xlabel("Data Points")
```

```
plt.ylabel("Distance")
```

```
plt.show()
```

Linkage methods:

- **Single:** Minimum distance between clusters.
- **Complete:** Maximum distance.
- **Average:** Average distance.

Different linkage methods can produce different cluster structures.

14. Artificial Neural Network with Backpropagation

```
import numpy as np
```

```
X = np.array([[0,0], [0,1], [1,0], [1,1]])
```

```
y = np.array([[0], [1], [1], [0]])
```

```
np.random.seed(1)
```

```
W1 = np.random.randn(2, 3)
```

```
W2 = np.random.randn(3, 1)
```

```
lr = 0.5
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```

for epoch in range(10000):

    # Forward propagation
    h = sigmoid(X @ W1)
    output = sigmoid(h @ W2)

    # Backpropagation
    error = y - output
    d_output = error * output * (1-output)

    d_hidden = (d_output @ W2.T) * h * (1-h)

    W2 += lr * h.T @ d_output
    W1 += lr * X.T @ d_hidden

prediction = sigmoid(sigmoid(X @ W1) @ W2)

print("Predicted Outputs:")
print(prediction.round(2))

```

Effect of parameters:

- More hidden neurons can learn complex patterns.
- Too few neurons may cause underfitting.
- Higher learning rate can speed learning but may become unstable.

15. Apriori Algorithm

```

from mlxtend.frequent_patterns import apriori, association_rules

import pandas as pd

transactions = [
    ['Milk', 'Bread'],

```

```
['Milk', 'Bread', 'Butter'],  
['Bread', 'Butter'],  
['Milk', 'Butter'],  
['Milk', 'Bread', 'Butter']  
]
```

```
items = sorted(set(item for t in transactions for item in t))
```

```
data = []
```

```
for t in transactions:
```

```
    data.append([1 if item in t else 0 for item in items])
```

```
df = pd.DataFrame(data, columns=items)
```

```
frequent = apriori(df, min_support=0.4, use_colnames=True)
```

```
rules = association_rules(  
    frequent,  
    metric="confidence",  
    min_threshold=0.6  
)
```

```
print("Frequent Itemsets:")
```

```
print(frequent)
```

```
print("\nAssociation Rules:")
```

```
print(rules[['antecedents', 'consequents',  
            'support', 'confidence', 'lift']])
```

Important measures

Support:

Confidence:

Lift:

Higher minimum support/confidence → fewer rules.

Lower thresholds → more rules, but potentially less useful rules.